
METALOGOS

Full Project Report

M1 through Phase 1: Fluid Types

Language: Rust (pest + tree-walking interpreter)

Repository: github.com/ShkodnikAI/Metalogos

Date: 2026-05-31

Version: 0.1.0

Table of Contents

1. Introduction	4
2. M1 -- Core: Entity, Pattern, Flow	5
2.1 Goal	5
2.2 Contract Program	5
2.3 Architecture Decisions	5
2.4 Files Modified	5
3. M2 -- Confidence, Rules, Flow Branching	6
3.1 Goal	6
3.2 Contract Program	6
3.3 Architecture Decisions	6
4. M3 -- Learnable Patterns and LLM Integration	7
4.1 Goal	7
4.2 Contract Program	7
4.3 Architecture Decisions	8
5. M4 -- Memory: Memorize, Recall, Forget, Decay	8
5.1 Goal	8
5.2 Contract Program	8
5.3 Architecture Decisions	8
6. M5 -- Adapt: Few-Shot Self-Modification	9
6.1 Goal	9
6.2 Contract Program	9
6.3 Architecture Decisions	9
7. Phase 1 -- Fluid Types: Lazy Collapse	10
7.1 Goal	10
7.2 Contract Program	10

7.3 Architecture Decisions	10
8. Architecture Overview	11
9. Architecture Decision Records	11
10. Growth Path and Roadmap	12

1. Introduction

METALOGOS is the first programming language designed by AI for AI. The name derives from Greek meta (beyond) + logos (reason, word, law) -- a language beyond human logic, where learning is a natural operation, data is primary and code is secondary, and self-modification is a feature, not a bug. Unlike conventional languages created by humans for humans, METALOGOS treats uncertainty as a first-class concept, memory as a semantic store with decay, and adaptation as a safe runtime operation.

The language is built around seven pillars: Entity (typed data with identity), Pattern (learnable transformations), Flow (declarative data pipelines), Memory (semantic storage with decay), Rule (probabilistic conditionals), Learn (LLM-backed pattern execution), and Adapt (safe self-modification via few-shot accumulation). Each pillar was implemented incrementally through a contract-first development methodology: for every feature, a contract program (.mlog + .expected) was written first, then the minimum runtime code to make it pass, followed by an Architecture Decision Record (ADR) documenting the design rationale.

This report covers all completed milestones: M1 (core loop), M2 (confidence and rules), M3 (LLM-backed learnable patterns), M4 (memory with decay), M5 (adapt via few-shot accumulation), and Phase 1 (Fluid Types with lazy collapse). Every milestone contributes exactly one new language capability while keeping all previous tests green. The total test suite consists of 6 golden tests that collectively validate the entire language specification implemented to date.

Miles tone	Feature	Contract Output	Status
M1	Entity + Pattern + Flow	HELLO, METALOGOS!!	Green
M2	Rules + Branching + Struct Entities	ESCALATE	Green
M3	LLM-backed Learnable Patterns	Response: complaint	Green
M4	Memory: Memorize + Recall + Decay	user likes spicy food	Green
M5	Adapt: Few-shot Self-modification	Hello, world!	Green
P1	Fluid Types: Lazy Collapse	84	Green

Table 1. Milestone summary and test status

2. M1 -- Core: Entity, Pattern, Flow

2.1 Goal

M1 proved that the lexer-parser-AST-interpreter loop closes end-to-end. The contract program defines a simple entity, a pure pattern with string manipulation, and a linear flow pipeline. When executed, it prints "HELLO, METALOGOS!!" -- demonstrating that the fundamental execution model works. This milestone established the project structure, the pest PEG grammar, the AST representation, the tree-walking interpreter, and the golden test harness that all subsequent milestones rely on.

2.2 Contract Program

```
entity greeting: String = "Hello, Metalogos!"
pattern Shout(s: String) -> String { return upper(s) + "!" }
flow Main { input: String = greeting -> Shout -> output }
```

2.3 Architecture Decisions

Parser (pest 2.x): Chosen over chumsky for its declarative grammar file that doubles as documentation. The PEG semantics are intuitive for a small language. Key lesson: inline string literals in pest do not produce pairs in `into_inner()`; named atomic rules do. All operators must be defined as named atomic rules for the AST converter to extract them reliably.

Interpreter (tree-walking): A direct AST evaluator without bytecode or JIT. Entities stored in `HashMap<String, Value>`. Patterns compiled into `CompiledPattern` structs (params + body). Flows executed as linear pipelines: evaluate source, thread through pipeline steps by invoking patterns/builtins, return final value. Performance optimization explicitly deferred to Phase 4 per the Agent Charter.

Multi-character operator ->: Conflicts with subtraction operator "-". Solved with `ARROW` as an atomic rule that matches before "-", and negative lookahead in `binop` to prevent "-" from matching when followed by ">".

2.4 Files Modified

File	Purpose
grammar.pest	PEG grammar: entity, pattern, flow, expressions

ast.rs	AST types: EntityDecl, PatternDecl, FlowDecl, Expr, Statement
parser.rs	pest pairs to AST conversion
interpreter.rs	Tree-walking evaluator: env, patterns, flow execution
builtins.rs	Built-in functions: upper, lower, len, str, print
lib.rs / main.rs	Public API (run_program) + CLI (mlog run)
tests/golden.rs	Golden test runner: auto-discovers .mlog/.expected pairs

Table 2. Files created in M1

3. M2 -- Confidence, Rules, Flow Branching

3.1 Goal

M2 introduced three capabilities that make the language feel probabilistic: struct entities with typed fields and defaults, rules with conditions and priority ordering, and flow branching by confidence thresholds. The contract demonstrates a message triage system: a rule detects urgency keywords and elevates the urgency score, then the flow dispatches to different response patterns based on the urgency level.

3.2 Contract Program

```
entity Message { text: String, urgency: Float = 0.0 }
entity m: Message = { text: "...", urgency: 0.0 }
rule If(m.text contains "...") then m.urgency = 0.9 with priority=10
flow Main {
  input: Message = m -> Classify -> output
  Classify {
    high (m.urgency > 0.8) -> Escalate
    medium (m.urgency < 0.8) -> Queue
    low (m.urgency < 0.4) -> Ignore
  }
}
```

3.3 Architecture Decisions

Rule engine (priority-ordered, first-wins): Rules are sorted by priority descending; stable sort preserves declaration order for equal priority. First matching rule wins -- no chaining, no conflict detection. Grounded in production systems (Rete, CLIPS) but

deliberately simplified to single-pass application. Forward-chaining to fixpoint and weighted inference (Markov Logic Networks style) are deferred to later milestones.

Flow architecture (pipeline + branch_def): A fundamental restructuring where the pipeline is purely linear (sequence of step names from input to output) and branch definitions are separate named blocks that follow the pipeline. This separation provides clean PEG parsing, orthogonal extensibility, and clear semantics: the pipeline is a dispatch chain, branch definitions are pattern-matching tables.

Confidence propagation (honest): M2 does not implement a full Fluid Type system. Float fields on struct entities serve as explicit confidence values, set by rules, read by branch conditions, used to route flow execution. This is documented as a deliberate simplification -- not probabilistic reasoning, but deterministic field assignment with numeric thresholds.

4. M3 -- Learnable Patterns and LLM Integration

4.1 Goal

M3 introduced the first "wow" moment of METALOGOS: a learnable pattern is executed by sending its prompt to an LLM and returning the model response as a typed value. This transforms METALOGOS from a pure-functional pipeline language into an AI-native programming language where patterns can invoke external intelligence. The contract demonstrates text classification: a learnable pattern with a "complaint" prompt receives input text, classifies it, and passes the result to a response pattern.

4.2 Contract Program

```
entity text: String = "..."  
learnable pattern Classify(msg: String) -> String {  
  prompt: "complaint"  
}  
pattern Respond(category: String) -> String {  
  return "Response: " + category  
}  
flow Main { input: String = text -> Classify -> Respond -> output }
```

4.3 Architecture Decisions

LLM backend abstraction (trait-based): The LlmBackend trait provides two implementations: MockLlm (test mode, returns prompt as deterministic response for golden tests) and RealLlm (production mode, HTTP POST via curl to configurable API endpoint supporting OpenAI and Ollama JSON formats). Mode selection via environment variable METALOGOS MOCK_LLM (default: true for safety).

Interpreter integration: Learnable patterns stored in a separate HashMap with highest dispatch priority (checked before builtins and pure patterns). Arguments concatenated into input string, prompt + input sent to backend, response returned as Value::String. Confidence propagation and structured output parsing are deferred.

5. M4 -- Memory: Memorize, Recall, Forget, Decay

5.1 Goal

M4 introduced persistent memory to METALOGOS programs. Until M4, all state was confined to entities and variables within a single execution. M4 adds the ability to store facts in memory (memorize), retrieve them by substring similarity with activation-based ranking (recall), and remove them by age (forget). The memory model is grounded in ACT-R base-level learning with exponential decay.

5.2 Contract Program

```
memorize "user likes spicy food" with priority=0.9
memorize "user hates cold soup" with priority=0.7
pattern FindFood(query: String) -> String {
  return recall(query)
}
flow Main { input: String = "spicy" -> FindFood -> output }
```

5.3 Architecture Decisions

Memory model (ACT-R inspired): Each entry stores value, priority, timestamp, and decay_rate. Activation = priority * exp(-decay_rate * age_in_days). Recall filters entries by substring match, ranks by activation, returns highest-activation entry above min_confidence threshold. No match returns empty string (soft-failure).

Forget (time-based removal): Removes entries matching query string with timestamp older than now - (days * 86400). Irreversible within execution. Default decay_rate = 0.01 (effectively no decay in millisecond-length golden tests). Vector similarity and embedding-based recall are deferred to Phase 2.

6. M5 -- Adapt: Few-Shot Self-Modification

6.1 Goal

M5 introduces the first form of self-modification to METALOGOS: an adapt declaration adds few-shot examples to a learnable pattern at runtime. When the pattern is subsequently invoked, it checks the few-shot examples first (exact-match cache) and returns the cached output without calling the LLM. This is the foundation for in-context learning -- the simplest defensible form of program self-modification.

6.2 Contract Program

```
learnable pattern Greet(name: String) -> String {  
  prompt: "hello"  
}  
adapt Greet add_example("world", "Hello, world!")  
pattern RunGreet(input: String) -> String {  
  return Greet(input)  
}  
flow Main { input: String = "world" -> RunGreet -> output }
```

6.3 Architecture Decisions

Safety invariant: Adapt only modifies learnable patterns -- it cannot modify pure patterns, rules, entities, or flows. Enforced at the AST level (AdaptDecl only references a pattern name, interpreter only looks up learnable_patterns). This is the key safety invariant that distinguishes METALOGOS adapt from arbitrary self-modification.

What M5 defers: mutate pattern with rollback_if, sandbox execution with allow/forbid lists and timeout, and adapt with new_example feedback syntax. These require test-suite runners, accuracy metrics, and capability-based sandboxing -- documented in ADR 0005 as the explicit roadmap for post-M5 development.

7. Phase 1 -- Fluid Types: Lazy Collapse

7.1 Goal

Phase 1 introduces the first type-level uncertainty to METALOGOS. A fluid value is a superposition of typed variants, each annotated with a confidence score. The superposition is materialized eagerly (all variant values computed at declaration), but the choice of which variant to use is deferred -- lazy collapse happens only at the point of use, when a typed context requires a specific type. This follows the metalogos-language-semantics skill guidance: "Fluid = tagged union of variants + confidence vector. Collapse is lazy, at the point of use."

7.2 Contract Program

```
fluid x = Float[42.0][0.9] or String["answer"][0.1]
pattern Double(n: Float) -> Float { return n + n }
flow Main { input: Float = x -> Double -> output }
```

Execution trace: x is declared as a fluid superposition with two variants: Float[42.0] at confidence 0.9 and String["answer"] at confidence 0.1. The flow passes x to Double, which requires a Float parameter. This triggers lazy collapse: the Float variant with confidence 0.9 is selected (above threshold 0.1), yielding 42.0. Double computes $42.0 + 42.0 = 84$. Output: "84".

7.3 Architecture Decisions

Syntax: fluid name = TypeName[value][confidence] or ... Each branch specifies type name, concrete value, and confidence (0.0..1.0). Multiple branches separated by "or".

Runtime: Value::Fluid(Vec<FluidValueVariant>) where each variant holds an already-evaluated concrete Value alongside its type name and confidence. Display impl shows highest-confidence variant value.

Lazy collapse: Triggered at pattern invocation via bind_and_collapse and maybe_collapse. For each (param, arg) pair, if arg is Fluid and param has a declared type, the interpreter finds the best matching variant by type + confidence. If confidence $\geq$ COLLAPSE_THRESHOLD (0.1), returns concrete value; otherwise returns Unit (soft-failure). Non-Fluid values pass through unchanged -- zero overhead.

What Phase 1 defers: Confidence propagation (output confidence from input confidence), automatic type coercion between variants, probabilistic type inference,

and Fluid-to-Fluid binary operations. These are documented in ADR 0006 as the growth path for subsequent Phase 1 iterations.

8. Architecture Overview

The current METALOGOS implementation is a single-crate Rust project with a tree-walking interpreter. The compilation pipeline is: `.mlog source --> pest parser --> AST --> interpreter --> stdout`. There is no bytecode, no JIT, no separate compilation phase. This is intentional for the MVP -- performance optimization is explicitly deferred to Phase 4 per the Agent Builder Charter.

Component	Module	Responsibility
Grammar	grammar.pest	PEG grammar for all language constructs
AST	ast.rs	Declaration, Expr, Statement, Branch types
Parser	parser.rs	pest pairs to AST conversion
Interpreter	interpreter.rs	Tree-walking eval, collapse, flow engine
LLM Client	llm.rs	LlmBackend trait, MockLlm, RealLlm
Built-ins	builtins.rs	upper, lower, len, str, print, contains, float
Public API	lib.rs	run_program() entry point
CLI	main.rs	mlog run <file.mlog>
Tests	tests/golden.rs	Auto-discovers .mlog/.expected pairs

Table 3. Current module architecture

9. Architecture Decision Records

Every milestone produces an ADR documenting the design rationale, prior art, decisions made, alternatives considered, and consequences. ADRs are the authoritative source for understanding why the language works the way it does.

AD R	Title	Milestone	Key Decision
0001	M1 Architecture	M1	pest PEG + tree-walking interpreter
0002	Rule Engine	M2	Priority-ordered, first-wins conflict resolution
0003	Learnable Semantics	M3	Trait-based LLM abstraction with mock
0004	Memory	M4	ACT-R decay, substring recall, soft-failure
0005	Adapt	M5	Few-shot only, no arbitrary code modification
0006	Fluid Types	P1	Lazy collapse, threshold 0.1, soft-failure

Table 4. Architecture Decision Records index

10. Growth Path and Roadmap

METALOGOS development follows a strict milestone-based approach. Each milestone adds exactly one new capability while maintaining backward compatibility. The roadmap is organized into four phases beyond the current M1-P1 milestones.

Phase	Scope	Key Features
Phase 1 (current)	Type System	Fluid Types, confidence propagation, Entity Store, codegen
Phase 2	ML Integration	PyO3/PyTorch/ONNX, knowledge graph, vector memory, transfer learning
Phase 3	Ecosystem	CLI/REPL, LSP, mlogpkg, standard library, mdbook
Phase 4	Performance	Bytecode VM/JIT, optimization passes, self-hosting compiler

Table 5. Development roadmap phases